5 Příkazy

V této kapitole se dozvíte:

- Jak zapsat větvení v PHP.
- Jaké příkazy cyklu je možné použít v PHP.
- Jak vložit do PHP souboru obsah jiného souboru.
- Jak opustit stránku v PHP.

Po jejím prostudování byste měli být schopni:

- Umět vytvořit podmínku v PHP pomocí příkazu if…else.
- Znát jednotlivé druhy cyklu, které jsou dostupné v PHP.
- Umět pracovat s příkazy require a include.
- Umět opustit stránku v PHP.

Klíčová slova této kapitoly:

if…else, while, do…while, for, break, continue, require, include, exit.

Doba potřebná ke studiu: 6 hodin



Průvodce studiem

Tato kapitola seznamuje studenty se základními řídícími strukturami jazyka PHP. Jedná se o podmínky, cykly, příkazy pro vkládání obsahu jiných souborů a příkazy pro opuštění stránky. Pokud máte základní znalosti algoritmizace, nebude pro Vás studium této kapitoly náročné. Pokud tyto znalosti nemáte, musíte sáhnout po jiné literatuře, protože základy algoritmizace nejsou předmětem této studijní opory.

Příkazy tvoří kostru aplikace. Se základními příkazy jsme se již seznámili v předchozích kapitolách. Jedná se příkaz pro výpis textu do výstup, přiřazení hodnoty do proměnné, definování konstanty nebo provedení výrazu. Kromě těchto příkazů, které se provádějí sekvenčně ve stejném pořadí v jakém jsou uvedeny ve zdrojovém kódu, můžeme v PHP používat příkazy pro větvení, opakování části kódu nebo pro vkládání obsahu jiných souborů do zdrojového kódu. S těmito příkazy se seznámíme v této kapitole.

5.1 Větvení programu

Pomocí větvení programu můžeme vytvořit části (bloky) kódu, které se provedou pouze za určitých podmínek. K větvení jsou v PHP k dispozici dva příkazy – první je příkaz `if…else` a druhým příkaz `switch`.



5.1.1 Příkaz if

Základním příkazem pro větvení programu v PHP je příkaz `if`. Tento příkaz je dostupný prakticky v každém programovacím jazyce. Syntaxe příkazu `if`

v PHP je stejná jako v Javě a je možné pomocí něho vytvořit neúplné, úplné nebo vnořené větvení.

Výpis 5.1 Syntaxe příkazu if.

```
<?php
  // neúplné větvení
  if (podmínka) {
    // příkazy
  }

  // úplné větvení
  if (podmínka) {
    // příkazy
  } else {
    // příkazy
  }

  // vnořené větvení
  if (podmínka1) {
    // příkazy
  } elseif (podmínka2) {
    // příkazy
  } else {
    // příkazy
  }
?>
```

Složené závorky nemusí být uvedeny, pokud má být v dané části proveden pouze jeden příkaz. V případě, že má být provedeno více příkazů, je nutné složené závorky uvést. Podmínka uvedená v části `if` nebo `elseif` musí být logickým výrazem, výsledkem tohoto výrazu musí být hodnota `true` nebo `false`.

Na počátku provádění příkazu `if` se vždy nejprve vyhodnotí podmínka uvedená v závorkách u příkazu `if`. Pokud je použito neúplné větvení a podmínka je vyhodnocena jako `true`, provede se příkaz (příkazy) následující za příkazem `if`. Pokud je podmínka vyhodnocena jako `false`, nestane se nic. V případě úplného větvení je začátek provádění příkazu `if` stejný. Pokud je výsledkem vyhodnocení podmínky `true`, provedou se příkazy v části `if`, pokud je výsledkem `false`, provedou se příkazy v části `else`. U vnořeného větvení se nejprve vyhodnotí podmínku uvedená v závorkách u příkazu `if`, tj. podmínka1. Pokud je výsledkem `true`, provedou se příkazy následující za ní.

Pokud je výsledkem `false`, vyhodnotí se podmínka v části `elseif`, tj. podmínka2. Pokud je podmínka2 vyhodnocena jako `true`, jsou provedeny příkazy následující za ní. Část `elseif` se může vyskytovat vícekrát, pokud je podmínka uvedená u příslušné části `elseif` vyhodnocena jako `false`, pokračuje se v následující části `elseif`. Pokud je podmínka u poslední části `elseif` vyhodnocena jako `false`, provedou se příkazy uvedené v části `else`.

Výpis 5.2 Příklad neúplného větvení.



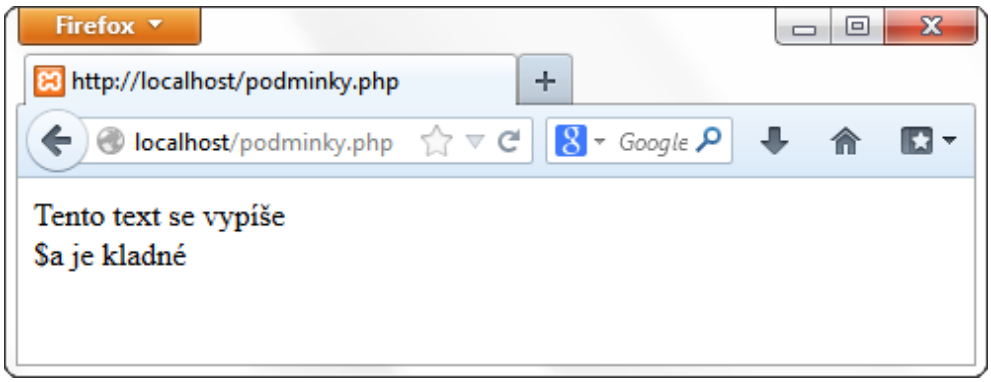
```
<?php
  if (false) echo "Tento text se nevypíše<br>";

  if (true) echo "Tento text se vypíše<br>";

  $a = 10;
  if ($a > 0) echo "\$a je kladné<br>";

  if ($a < 0) echo "\$a je záporné<br>";
?>
```

Výsledkem vykonání uvedených příkazů bude výstup zobrazený na následujícím obrázku.



Obrázek 5.1 Neúplné větvení.

Úplné větvení obsahuje kromě části `if` také část `else`.

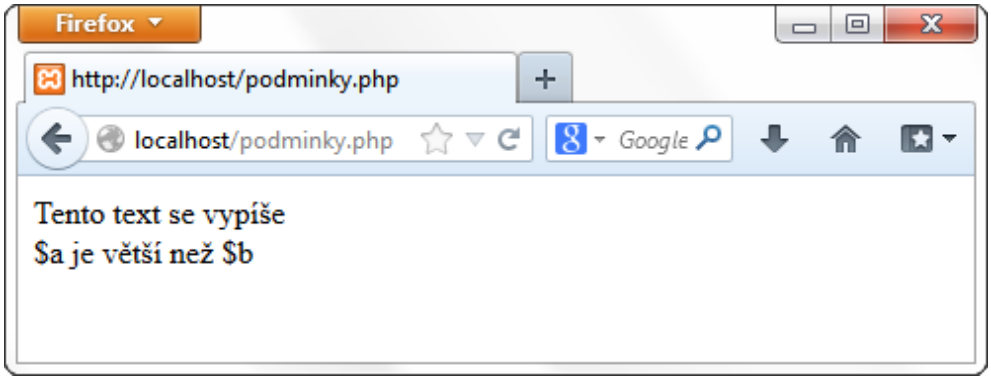
Výpis 5.3 Příklad úplného větvení.



```
<?php
  if (true) {
    echo "Tento text se vypíše<br>";
  } else {
    echo "Tento text se nevypíše<br>";
  }
```

```
  $a = 10;
  $b = 5;
  if ($a > $b) {
    echo "\$a je větší než \$b<br>";
  } else {
    echo "\$a je menší než \$b<br>";
  }
?>
```

Výsledný výstup zobrazený v prohlížeči je zobrazen na následujícím obrázku.



Obrázek 5.2 Úplné větvení.

Vnořené větvení obsahuje kromě částí `if` a `else` navíc jednu nebo více části `elseif`. Následující výpis obsahuje ukázku dvou vnořených větvení a obrázek 5.3 výstup zobrazený v prohlížeči.

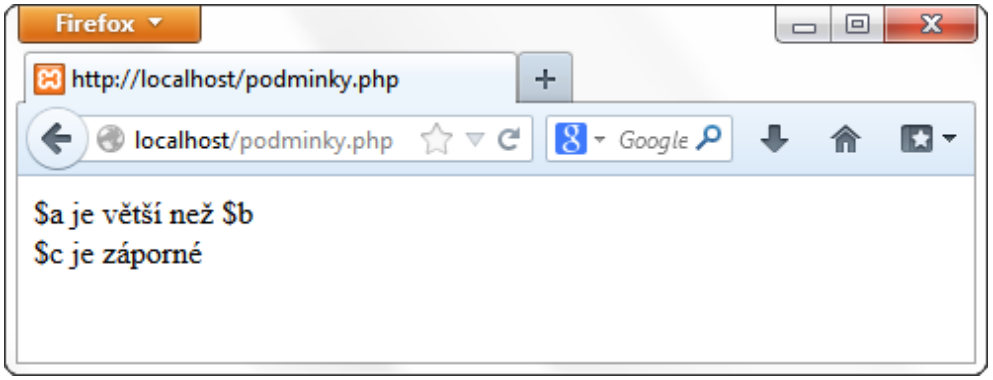
Výpis 5.4 Ukázka vnořeného větvení.



```
<?php
  $a = 10;
  $b = 5;
  if ($a > $b) {
    echo "\$a je větší než \$b<br>";
  } elseif ($a == $b) {
    echo "\$a je rovno \$b<br>";
  } else {
    echo "\$a je menší než \$b<br>";
  }


  $c = -3;
  if ($c > 0) {
    echo "\$c je kladné<br>";
  } elseif ($c == 0) {
```

```
    echo "\$c je rovno 0<br>";
  } else {
    echo "\$c je záporné<br>";
  }
?>
```



Obrázek 5.3 Výstup zobrazený v prohlížeči.

Jazyk PHP umožňuje pro vnořené větvení alternativní syntaxi. Za podmínkami jsou uvedeny dvojtečky a není potřeba uvádět složené závorky pro blok příkazů. Příkaz if je ukončen příkazem endif;. Tento způsob syntaxe je užitečný v případě, kdy potřebujeme vložit do výstupu HTML kód.

Výpis 5.5 Alternativní syntaxe vnořeného větvení.

```
<?php
  if (podmínka1):
    // příkazy
  elseif (podmínka2):
    // příkazy
  elseif (podmínka3):
    // příkazy
  elseif (podmínka4):
    // příkazy
  endif;
?>
```

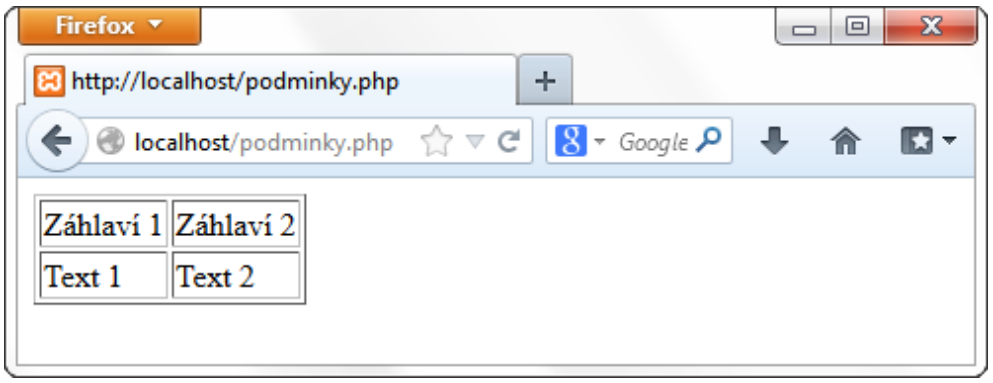
Konkrétní příklad použití alternativní syntaxe pro vnořené větvení je uveden v následujícím výpisu, výstup zobrazený v prohlížeči je zobrazen na obrázku pod ním.

Výpis 5.6 Konkrétní příklad použití alternativní syntaxe.



```
<?php
  $text = "tabulka";
  if ($text == "tabulka"):
?>
```

```
  <table border="1">
    <tr>
      <td>Záhlaví 1</td><td>Záhlaví 2</td>
    </tr>
    <tr>
      <td>Text 1</td><td>Text 2</td>
    </tr>
  </table>
<?php elseif ($text == "obrazek"): ?>
  <img src="obrazek" atl="">
<?php elseif ($text == "odstavec"): ?>
  <div>Odstavec</div>
<?php elseif ($text == "nadpis"): ?>
  <h1>Nadpis</h1>
<?php endif; ?>
```



Obrázek 5.4 Výstup použití alternativní syntaxe zobrazený v prohlížeči.

5.1.2 Příkaz switch



Druhým příkazem, pomocí něhož můžeme v PHP realizovat větvení programu, je příkaz `switch`. Jeho syntaxe je opět velmi podobná syntaxi příkazu `switch` v Javě. Následující výpis představuje obecnou syntaxi příkazu `switch`.

Výpis 5.7 Obecná syntaxe příkazu switch.

```
<?php
  switch(proměnná) {
    case návěští1:
      // příkazy1
    break;
    case návěští2:
      // příkazy2
    break;
    case návěští3:
      // příkazy4
```

```
    break;
    default:
      // příkazy5
  }
?>
```

V závorkách za příkazem `switch` následuje identifikátor proměnné, pomocí jejíž hodnoty chceme provést větvení. Pokud hodnota této proměnné odpovídá některému z návěští uvedených v částech `case`, provedou se příkazy, které následují za touto částí. Příkazy se provádí až do příkazu `break`. Pokud neodpovídá hodnota proměnné žádnému z návěští u částí `case`, provedou se příkazy v části `default`. Tato část je nepovinná a pokud je umístěna ve výčtu možností až jako poslední, není nutné za ní uvádět příkaz `break`, protože je ukončena složenou závorkou, která ukončuje celý příkaz `switch`. V některých případech je možné použít i syntaxi, která je uvedena v následujícím výpisu.

Výpis 5.8 Jiná možnost použití příkazu switch.

```
<?php
  switch(proměnná) {
    case návěští1: // zde chybí break;
    case návěští2:
      // příkazy1
    break;
    case návěští3: // zde chybí break;
    case návěští4:
      // příkazy2
    break;
    default:
      // příkazy3
  }
?>
```

Jak vidíme ve výpisu, u některých návěští chybí příkaz `break`. Použití tohoto kódu nahrazuje použití logické spojky `or` v příkazu `if`. Pokud bude hodnota proměnné rovna `návěští1` nebo `návěští2`, provedou se `příkazy1`. Pokud bude hodnota proměnné rovna `návěští3` nebo `návěští4`, provedou se `příkazy2`. V případě, že hodnota proměnné nebude rovna žádnému z návěští, provedou se `příkazy3`.

Proměnná, pomocí jejíž hodnoty se provádí větvení, může být jakéhokoliv jednoduchého datového typu, tj. celé číslo, desetinné číslo nebo řetězec.

Výpis 5.9 Ukázka použití příkazu switch.

```
<?php
  $jazyk = "cz";
  switch($jazyk) {
    case "en":
      echo "Angličtina";
    break;
    case "cz":
      echo "Čeština"; // toto se vypíše
    break;
    case "sk":
      echo "Slovenština";
    break;
    case "fr":
      echo "Francouzština";
    break;
    default:
      echo "Jiný jazyk";
  }
?>
```



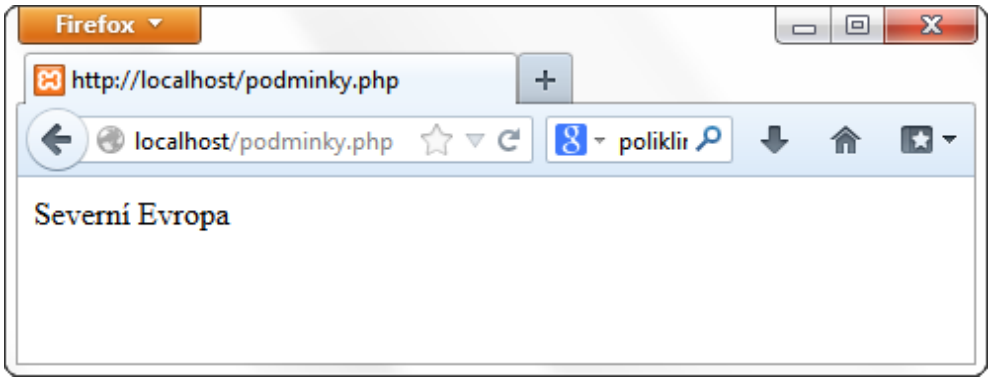
Následující výpis představuje použití varianty příkazu `switch`, kdy příkaz `break` není uveden u všech návěští.

Výpis 5.10 Ukázka použití příkazu switch nahrazujícího spojku "nebo".

```
<?php
  $zeme = "Švédsko";
  switch($zeme) {
    case "Velká Británie":
    case "Německo":
    case "Francie":
      echo "Západní Evropa";
    break;
    case "Česká republika":
    case "Slovenská republika":
      echo "Střední Evropa";
    break;
    case "Finsko":
    case "Švédsko":
    case "Norsko":
      echo "Severní Evropa";
    break;
    default:
      echo "Jiná část světa";
  }
```



```
?>
```



Obrázek 5.5 Výstup zobrazený v prohlížeči.

5.2 Cykly

Pro opakování stejné činností vícekrát po sobě slouží v programování cykly. I v tomto případě jsou cykly v jazyce PHP velmi podobné jako v jazyce Java. Jazyk PHP umožňuje použít čtyři různé cykly: `while`, `do…while`, `for` a `foreach`.

5.2.1 Cyklus while



Cyklus `while` představuje cyklus s podmínkou na začátku. Nejprve je otestována podmínka, která musí být ve tvaru logického výrazu. Pokud je podmínka vyhodnocena jako `true`, provede se příkaz nebo příkazy v těle cyklu a znovu se testuje podmínka. Pokud je podmínka stále `true`, vykonávají se opakovaně příkazy uvedené v těle cyklu. Jakmile je podmínka vyhodnocena jako `false`, provádění cyklu je ukončeno a program pokračuje příkazy následujícími za cyklem `while`. U tohoto cyklu se může stát, že se neprovede ani jednou.

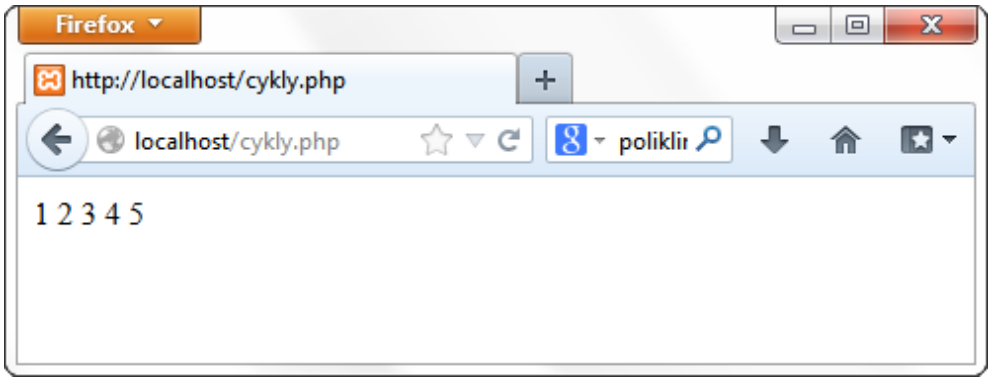
Výpis 5.11 Obecná syntaxe cyklu while.

```
<?php
  while (podmínka) {
    // příkazy
  }
?>
```

V následujícím výpisu je uvedena příklad použití cyklu `while`. Cyklus vypíše do výstupu čísla `1` až `5`.

Výpis 5.12 Ukázka použití cyklu while.

```
<?php
  $cislo = 1;
  while ($cislo <= 5) {
    echo $cislo . " ";
    $cislo++;
  }
?>
```



Obrázek 5.6 Výsledek použití cyklu while zobrazený v prohlížeči.

U příkazu `while` je možné použít alternativní syntaxi, podobně jako u příkazu `if`. Pro ukončení cyklu se používá příkaz `endwhile`. Následující výpisy představují obecnou alternativní syntaxi a konkrétní příklad použití této syntaxe – příklad vypíše čísla `1` až `5`.

Výpis 5.13 Obecná alternativní syntaxe cyklu while.

```
<?php
  while (podmínka):
    // příkazy
  endwhile;
?>
```

Výpis 5.14 Příklad použití alternativní syntaxe cyklu while.

```
<?php
  $cislo = 1;
  while ($cislo <= 5):
    echo $cislo . " ";
    $cislo++;
  endwhile;
?>
```



5.2.2 Cyklus do…while

Druhým cyklem, který si představíme, je cyklus `do…while`. Jedná se o cyklus s podmínkou na konci. Nejprve jsou provedeny příkazy v těle cyklu a poté je vyhodnocena podmínka. Pokud je jejím výsledkem `true`, znovu se provedou příkazy v těle cyklu a opět je vyhodnocena podmínka. Provádění cyklu pokračuje tak dlouho, dokud není podmínka není vyhodnocena jako `false`. Tento cyklus se vždy provede alespoň jednou.

Výpis 5.15 Obecná syntaxe cyklu do...while.

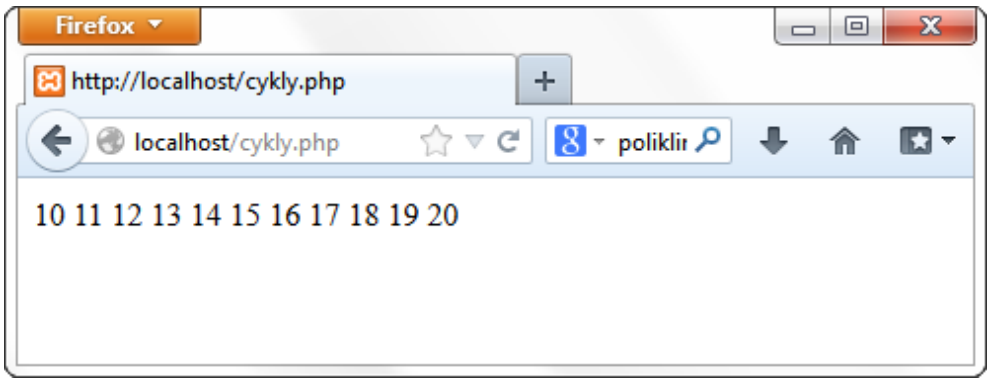
```
<?php
  do {
    // příkazy
  } while (podmínka);
?>
```

Následující výpis obsahuje příklad použití cyklu `do…while`. Tento příklad vypíše čísla od `10` do `20`.

Výpis 5.16 Příklad použití cyklu do...while.



```
<?php
  $cislo = 10;
  do {
    echo $cislo . " ";
    $cislo++;
  } while ($cislo < 21);
?>
```



Obrázek 5.7 Výsledek použití cyklu do...while.

5.2.3 Cyklus for



Dalším cyklem, který můžeme v PHP použít, je cyklus `for`. Tento cyklus bývá nazýván také jako cyklus s řídící proměnnou. Zatímco cykly `while` a `do…while` je možné použít i případě, kdy předem neznáme počet opakování,

která je potřeba provést, cyklus `for` je vhodný hlavně v případech, kdy počet opakování předem známe.

Výpis 5.17 Obecná syntaxe cyklu for.

```
<?php
  for (inicializace; podmínka; krok) {
    // příkazy
  }
?>
```



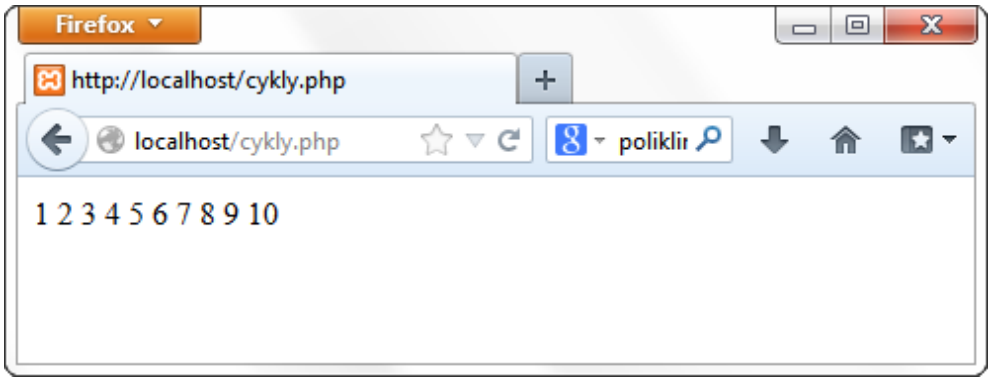
V závorce u příkazu jsou tři části oddělené navzájem znakem `;` (středník). První část zpravidla obsahuje inicializaci řídící proměnná, která určuje počet opakování cyklu. Tato část se provede pouze jednou před začátkem vykonávání cyklu. Druhou částí je `podmínka`, která určuje, kdy bude cyklus ukončen. Třetí část obsahuje krok, nejčastěji zde bývá příkaz pro inkrementaci nebo dekrementaci řídící proměnné. Po inicializaci se otestuje podmínka, pokud je vyhodnocena jako `true`, provedou se příkazy v těle cyklu. Poté se provedou příkazy ve třetí části a znovu se otestuje podmínka. Pokud je stále `true`, provedou se příkazy v těle cyklu a třetí část cyklu.

Následující výpis obsahuje příklad použití cyklu `for`, který vypíše do výstupu čísla do `1` do `10`.

Výpis 5.18 Konkrétní příklad použití cyklu for.

```
<?php
  for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
  }
?>
```

Na následujícím obrázku je zobrazen výsledek provedení tohoto cyklu zobrazený v prohlížeči.



Obrázek 5.8 Výsledek použití cyklu for.

Podobně jako příkazy `if` a `while`, i cyklus `for` umožňuje použití alternativní syntaxe. Tato syntaxe je uvedena v následujícím výpisu, k ukončení cyklu se používá příkaz `endfor`.

Výpis 5.19 Alternativní syntaxe příkazu for.

```
<?php
  for (inicializace; podmínka; krok):
    // příkazy
  endfor;
?>
```

Konkrétní příklad použití alternativní syntaxe je zobrazen v následujícím výpisu. Výstup tohoto výpisu je stejný jako ve výpisu 5.18.

Výpis 5.20 Příklad použití alternativní syntaxe cyklu for.



```
<?php
  for ($i = 1; $i <= 10; $i++):
    echo $i . " ";
  endfor;
?>
```

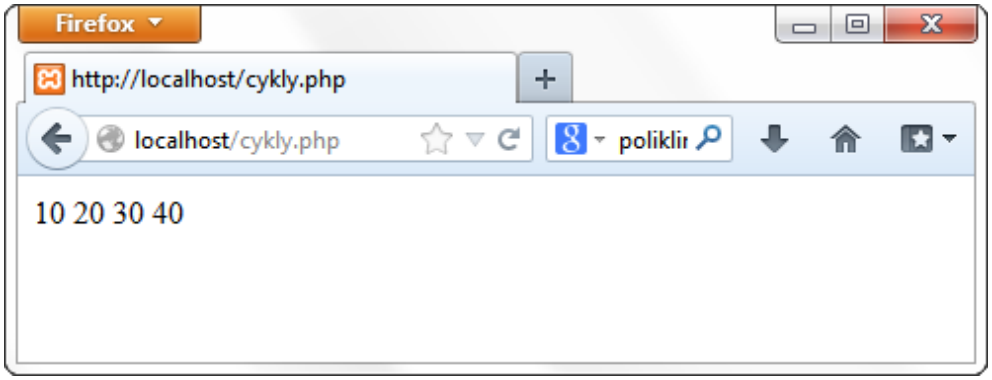
Typickým příkladem použití cyklu `for` je procházení spojitě indexovaného pole. Počet prvků pole je možné zjistit pomocí funkce `count()`, takže známe počet opakování, který je nutné provést, abychom prošli všechny prvky pole.

Výpis 5.21 Výpis spojitě indexovaného pole.



```
<?php
  $cisla = array(10, 20, 30, 40);
  for ($i = 0; $i < count($cisla); $i++) {
    echo $cisla[$i] . " ";
  }
?>
```

Výsledkem provedení tohoto cyklu je zobrazen na následujícím obrázku.



Obrázek 5.9 Obsah pole vypsaný v prohlížeči.

5.2.4 Cyklus foreach



Posledním cyklem, který zde uvedeme, je cyklus `foreach`. Tento cyklus se používá zejména při procházení nespojitě indexovaných polí nebo asociativních polí. Obecná syntaxe cyklu `foreach` může mít dvě podoby.

Výpis 5.22 Obecná syntaxe cyklu foreach.

```
<?php
  foreach (pole as $hodnota) {
    // příkazy
  }
  foreach (pole as $index => $hodnota) {
    // příkazy
  }
?>
```

V závorce za příkazem `foreach` je nejprve uveden výraz označující procházené pole. První varianta postupně prochází polem a každém kroku uloží hodnotu aktuálního prvku do proměnné `$hodnota`. Druhá varianta cyklu `foreach` navíc uloží v každém kroku index aktuálního prvku do proměnné `$index`. Poté se interní ukazatel pole posune na následující prvek. Pro procházení pole jsou navíc k dispozici funkce `reset()`, `next()`, `prev()`, `key()` a `current()`. Všechny tyto funkce mají jako parametr pole, se kterým pracujeme. Funkce `reset()` vrátí interní ukazatel v poli na začátek pole, tj. interní ukazatel se nastaví na první prvek pole. Funkce `next()` a `prev()` slouží k posunu interního ukazatele. Funkce `next()` posune interní ukazatel na následující prvek, tj. o jednu pozici doprava, funkce `prev()` jej posune na předchozí prvek, tj. o jednu pozici doleva. Obě funkce navíc vrátí hodnotu prvku, na který se posunuly. Pokud se pomocí této funkce posune

interní ukazatel mimo pole, funkce vrátí hodnotu `false`. Funkce `key()` vrací index aktuálního prvku v poli, na který ukazuje interní ukazatel. Funkce `current()` pak vrací hodnotu aktuálního prvku v poli.

Následující kód ukazuje použití obou variant cyklu `foreach` a na obrázku dále je zobrazen výstup zobrazený v prohlížeči. K vypsání polí vedle sebe je použita tabulka bez ohraničení.

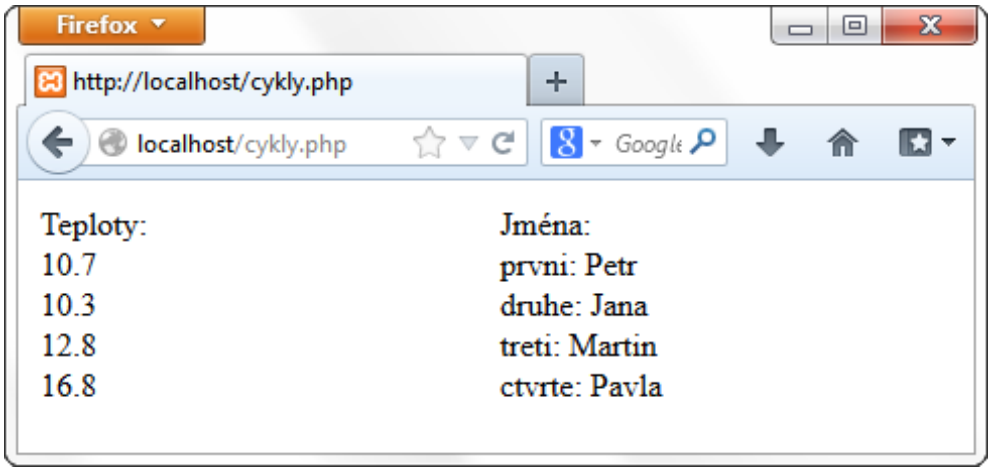
Výpis 5.23 Ukázka použití obou variant cyklu foreach.



```
<?php
  echo '<table border="0" width="100%"><tr><td
   width="50%">';
  // nespojitě indexované pole
  $teploty = array(0 => 10.7, 4 => 10.3, 8 => 12.8,
   12 => 16.8);
  echo "Teploty:<br>";
  foreach ($teploty as $hodnota) {
    echo $hodnota . "<br>";
  }

  echo '</td><td>';

  // asociativní pole
  $jmena = array("prvni" => "Petr", "druhe" =>
   "Jana", "treti" => "Martin", "ctvrte" => "Pavla");
  echo "Jména:<br>";
  foreach ($jmena as $index => $hodnota) {
    echo $index . ": " . $hodnota . "<br>";
  }
  echo '</td></tr></table>';
?>
```



Obrázek 5.10 Použití cyklu foreach.

5.2.5 Příkazy break a continue



Při používání cyklů je možné použít příkazy `break` a `continue`. Příkaz `break` ukončí provádění celého cyklu a vykonávání kódu pokračuje za cyklem. Příkaz `continue` ukončí aktuální iteraci cyklu a vrátí se na část vyhodnocení podmínky.

5.3 Vkládání souborů do stránek PHP



Jazyk PHP obsahuje dva příkazy, které načítají a provádějí kód ze zadaného souboru. Těmito příkazy jsou `require` a `include`. Použitím těchto příkazů je možné rozdělit aplikaci do více souborů, ve kterých budou umístěny společné konstanty nebo funkce. Vložením obsahu takového souboru pak můžeme použít konstantu nebo funkci na různých místech vytvářené aplikace. Celá aplikace je tím pádem přehlednější a lépe se s ní pracuje.

Příkaz `require` vloží obsah daného souboru na místo, kde je zavolán. Kód vkládaného souboru je tedy součástí kódu souboru, kde byl příkaz `require` zavolán. Protože příkaz `require` vkládá obsah daného souboru na místo, kde byl zavolán, není možné ho použít v cyklu pro načtení více různých souborů. K tomu je vhodnější použít příkaz `include`, který také zpřístupní kód z externího souboru, ale kód externího souboru se provádí při každém výskytu příkazu.

Pokud uvedeme u příkazu `require` nebo `include` pouze název souboru (např. `soubor.php`), hledá se daný soubor ve stejném adresáři, jako soubor, kde jsme tyto příkazy použili. Název souboru je tedy určen relativně. Samozřejmě je možné použít i absolutní cestu k souboru.

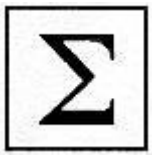
5.4 Opuštění stránky PHP

Posledním příkazem, který si v této kapitole uvedeme, je příkaz `exit`. Tento příkaz slouží k opuštění stránky PHP a k zastavení jejího dalšího provádění. Funguje podobně jako příkaz `break`, s tím rozdílem, že funguje na dokument jako na celek a ne pouze na danou část (například na cyklus). Příkaz `exit` je vhodné použít například při práci s databází, pokud dojde k chybě v komunikaci s databázovým serverem.



Kontrolní otázky:

1. K čemu slouží v PHP příkaz `if`?
2. Jaké příkazy cyklu můžeme použít v PHP?
3. K čemu slouží příkazy `break` a `continue`?
4. Jaký je rozdíl mezi příkazy `require` a `include`?
5. K čemu slouží příkaz `exit`?



Shrnutí obsahu kapitoly

V této kapitole jste se v úvodu seznámili s příkazem pro vytvoření větvení v PHP. Dále jste se seznámili s jednotlivými typy příkazů cyklu v PHP a s rozdíly mezi nimi. Jazyk PHP umožňuje použít cyklus s podmínkou na začátku, cyklus s podmínkou na konci i cyklus s řídící proměnnou. Příkazy `require` a `include` umožňují vložit do souboru obsah jiného souboru. Příkaz `exit` umožňuje opustit právě prováděný PHP skript.